

Screenshot ทดสอบ

GET /products/{id}

The screenshot shows the Postman interface with a GET request to `http://localhost:8080/products/1`. The response is a JSON object representing a product.

```
1 {
2   "id": "1",
3   "name": "iPhone 15 Pro (673308029-0 SEC 1)",
4   "category": "Electronics",
5   "brand": "Apple",
6   "stock": 50,
7   "price": 39900.0,
8   "discountType": "MEMBER",
9   "discountedPrice": 35910.0
10 }
```

Response status: 200 OK, 13 ms, 243 B.

GET /products

The screenshot shows the Postman interface with a GET request to `http://localhost:8080/products`. The response is a JSON array containing two product objects.

```
1 [
2   {
3     "id": "1",
4     "name": "iPhone 15 Pro (673308029-0 SEC 1)",
5     "category": "Electronics",
6     "brand": "Apple",
7     "stock": 50,
8     "price": 39900.0,
9     "discountType": "MEMBER",
10    "discountedPrice": 35910.0
11  },
12  {
13    "id": "2",
14    "name": "MacBook Air M3",
15    "category": "Electronics",
16    "brand": "Apple",
17    "stock": 20,
18    "price": 49900.0,
19    "discountType": "NONE",
20    "discountedPrice": 49900.0
21  }
22 ]
```

Response status: 200 OK, 148 ms, 564 B.

POST /products

The screenshot shows the Postman application interface. On the left, there's a sidebar with 'COLLECTIONS' and 'ENVIRONMENTS'. The main panel displays a POST request to 'http://localhost:8080/products'. The request body is a JSON object:

```
{  "name": "Test Product",  "category": "Electronics",  "brand": "Test",  "stock": 10,  "price": 999.0,  "discountType": "NONE"}
```

. The response is a 200 OK status with a JSON body:

```
{  "id": "513c27f9-9442-4359-a37f-1a4db8dcd9a",  "name": "Test Product",  "category": "Electronics",  "brand": "Test",  "stock": 10,  "price": 999.0,  "discountType": "NONE",  "discountedPrice": 999.0}
```

. The status bar at the bottom indicates '200 OK · 62 ms · 250 B'.

DELETE /products/{id}

The screenshot shows the Postman application interface. On the left, there's a sidebar with 'COLLECTIONS' and 'ENVIRONMENTS'. The main panel displays a DELETE request to 'http://localhost:8080/products/3'. The request body is empty, indicated by the text 'This request does not have a body'. The response is a 200 OK status. The status bar at the bottom indicates '200 OK · 14 ms · 38 B'.

GET /products/category/{cat}

The screenshot shows the Postman interface with a GET request to `http://localhost:8080/products/category/Electronics`. The response is a 200 OK status with a 6 ms response time and 582 B of data. The response body is a JSON array of two product objects.

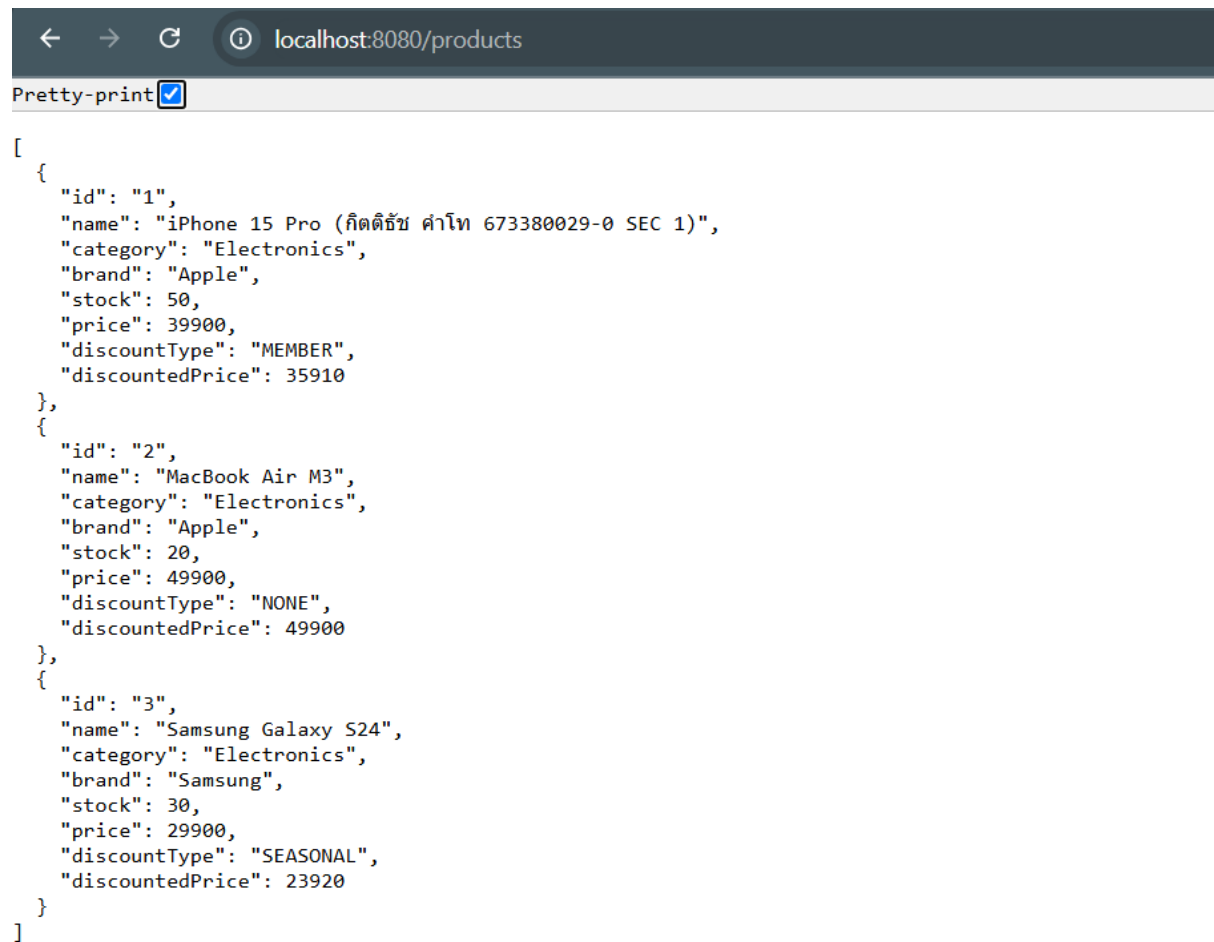
```
1 [
2   {
3     "id": "1",
4     "name": "iPhone 15 Pro (673388029-0 SEC 1)",
5     "category": "Electronics",
6     "brand": "Apple",
7     "stock": 56,
8     "price": 39900.0,
9     "discountType": "MEMBER",
10    "discountedPrice": 35916.0
11  },
12  {
13    "id": "2",
14    "name": "MacBook Air M3",
15    "category": "Electronics",
16    "brand": "Apple",
17    "stock": 28,
18    "price": 49900.0,
19    "discountType": "NONE",
20    "discountedPrice": 49900.0
21  }
22 ]
```

GET /products/{id}/price

The screenshot shows the Postman interface with a GET request to `http://localhost:8080/products/1/price`. The response is a 200 OK status with a 7 ms response time and 77 B of data. The response body is a single JSON object representing the price of the product with id '1'.

```
1 35916.0
```

ผลลัพธ์ใน response มีชื่อและรหัสนักศึกษา



```
[
  {
    "id": "1",
    "name": "iPhone 15 Pro (กิตติชัย คำโท 673380029-0 SEC 1)",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900,
    "discountType": "MEMBER",
    "discountedPrice": 35910
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900,
    "discountType": "NONE",
    "discountedPrice": 49900
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900,
    "discountType": "SEASONAL",
    "discountedPrice": 23920
  }
]
```

Reactive Programming vs Blocking — ต่างกันอย่างไร

Blocking คือโปรแกรมสั่งงานแล้ว Thread ต้องหยุดรอผลลัพธ์ ก่อนจึงจะทำงานต่อ ส่วน Reactive Programming คือ ไม่ต้องรอให้งานหนึ่งเสร็จก่อน ถึงจะไปทำงานอื่น

Mono vs Flux — ใช้กรณีไหน และทำไม

Mono ใช้ตอนมี 0 หรือ 1 ค่า และ Flux ใช้ตอนมีหลายค่าจาก collection

WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()

.get() คือต้องการส่ง HTTP GET Request .uri() คือจะส่ง Request ไปที่ URL ไหน .retrieve() คือส่ง Request แล้ว ตอนนี้ต้องการรับ Response กลับมา .bodyToMono/ Flux() คือ เอา Response Body ที่ได้มา แปลงเป็น Product แล้วห่อไว้ใน Mono<Product>/ Flux<Product>

Code ของ Repository, Service, Controller พร้อม comment อธิบาย

Repository

```
public Mono<Product> findById(String id) {  
    Product product = store.get(id);  
    if(product == null){  
        return Mono.empty(); // ← แก้มรรทัดนี้  
    }  
    return Mono.just(product);  
}
```

แสดง Product ตาม Id

Product product = store.get(id); ดึงข้อมูลจาก store ตาม Id มาใส่ใน product

if(product == null){ ถ้า product เป็น null จะ return กล่องเปล่า ถ้าไม่จะ return Mono ที่มี product

return Mono.empty();

}

return Mono.just(product);

```
public Flux<Product> findAll() {  
    Collection<Product> product = store.values();  
    return Flux.fromIterable(product) ;  
}
```

แสดง Product ทั้งหมด

Collection<Product> product = store.values(); ดึงส่วนข้อมูลของ store มาใส่ในรูป Collection<Product>

Flux.fromIterable(product) แปลง product เป็น Flux

```
public Mono<Product> save(Product product) {  
    // TODO: เติม code ตรงนี้  
    store.put(product.getId(), product);  
    return Mono.just(product); // ← แก้มรรทัดนี้  
}
```

เซฟข้อมูล

store.put(product.getId(), product); นำข้อมูลเพิ่มลง store

`Mono.just(product)`; คืนค่าเป็น `Mono` ที่มี `product`

```
public Mono<Void> deleteById(String id) {  
    // TODO: เติม code ตรงนี้  
    store.remove(id);  
    return Mono.empty(); // ← แก้มรทัดนี้  
}
```

ลบข้อมูลตาม ID

`store.remove(id)`; ลบข้อมูลใน `store`

`return Mono.empty()`; คืนค่าเป็น `Mono` ที่เป็นกล่องเปล่า

```
public Flux<Product> findByCategory(String category) {  
    // TODO: เติม code ตรงนี้  
    Flux<Product> product = findAll().filter(p -> p.getCategory().equalsIgnoreCase(category));  
    return product; // ← แก้มรทัดนี้  
}
```

หาข้อมูลตามหมวดหมู่

`Flux<Product> product = findAll().filter(p -> p.getCategory().equalsIgnoreCase(category))`; ดึงข้อมูลออกมาทั้งหมด แล้วกรองตามหาหมวดหมู่ตามที่ต้องการ หมวดหมู่ที่ค้นหาจะเป็นตัวพิมพ์ใหม่หรือเล็กก็ได้

`return product`; คืนค่า ข้อมูลที่เป็นแบบ `Flux`

Service

```
public Mono<Product> getById(String id) {  
    // TODO: เติม code ตรงนี้  
    return repository.findById(id).switchIfEmpty(Mono.error(new RuntimeException("Product not found: " + id))); // ← แก้มรทัดนี้  
}
```

แสดง Product ตาม Id

หาข้อมูลตาม id ถ้าไม่เจอจะแสดง `Product not found: + id`

```
public Flux<Product> getAll() {  
    // TODO: เติม code ตรงนี้  
    return repository.findAll(); // ← แก้มรทัดนี้  
}
```

แสดง Product ทั้งหมด

```
public Mono<Product> save(Product product) {
    // TODO: เติม code ตรงนี้
    if(product.getId() == null){
        product.setId(java.util.UUID.randomUUID().toString());
    }
    return repository.save(product); // ← แก้มรทัดนี้
}
```

บันทึกข้อมูล

if(product.getId() == null){ ถ้าเป็นข้อมูลใหม่ จะสร้าง ID อัตโนมัติ

```
product.setId(java.util.UUID.randomUUID().toString());
```

```
}
```

```
public Flux<Product> getByCategory(String category) {
    // TODO: เติม code ตรงนี้
    return repository.findByCategory(category); // ← แก้มรทัดนี้
}
```

แสดงข้อมูลตามหมวดหมู่

```
public Mono<Double> getDiscountedPrice(String id) {
    // TODO: เติม code ตรงนี้
    Mono<Double> a = getById(id).map(p -> p.getDiscountedPrice());
    return a; // ← แก้มรทัดนี้
}
```

คำนวณ

คำนวณราคาหลังส่วนลด

Controller

```
@GetMapping("/{id}")
public Mono<Product> getById(@PathVariable String id) {
    return service.getById(id);
}
```

HTTP Method เป็น GET ส่งต่อ id ไปให้ Service Layer เพื่อดึงข้อมูลสินค้าตาม id

```
@GetMapping
public Flux<Product> getAll() {
    // TODO: เติม code ตรงนี้
    return service.getAll(); // ← แก้มรทัดนี้
}
```

HTTP Method เป็น GET ส่งคำขอไป Service Layer เพื่อดึงข้อมูลสินค้าทั้งหมด

```
@PostMapping
public Mono<Product> save(@RequestBody Product product) {
    // TODO: เติม code ตรงนี้
    return service.save(product); // ← แก้มรทัดนี้
}
```

HTTP Method เป็น POST ส่งข้อมูลต่อไปให้ Service Layer เพื่อดึงข้อมูลสินค้าบันทึกสินค้า

```
@DeleteMapping("/{id}")
public Mono<Void> delete(@PathVariable String id) {
    // TODO: เติม code ตรงนี้
    return service.delete(id); // ← แก้มรทัดนี้
}
```

HTTP Method เป็น Delete ส่ง ID ต่อไปให้ Service Layer เพื่อลบข้อมูลสินค้า

```
@GetMapping("/category/{category}")
public Flux<Product> getByCategory(@PathVariable String category) {
    // TODO: เติม code ตรงนี้
    return service.getByCategory(category); // ← แก้มรทัดนี้
}
```

HTTP Method เป็น GET ส่งประเภทหมวดหมู่ต่อไปให้ Service Layer เพื่อดึงข้อมูลสินค้าตามหมวดหมู่

```
@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(@PathVariable String id) {
    // TODO: เติม code ตรงนี้
    return service.getDiscountedPrice(id); // ← แก้มรทัดนี้
}
```

HTTP Method เป็น GET ส่งคำขอต่อไปให้ Service Layer เพื่อให้ทำการค้นหาสินค้าและคำนวณราคา

